2-D Viewing and Clipping

Computer Graphics | Unit 2 | Classroom Notes

Viewing pipeline • Viewing coordinate frame • Window-to-viewport transformation • Viewing functions • Point clipping • Cohen–Sutherland line clipping • Sutherland–Hodgman polygon clipping

How to use these notes

- **Colour code:** blue = definition, amber = important point, green = example, red = common mistake, purple = activity, teal = real-world link.
- **Prerequisites:** 2-D transformations from the previous notes (translation, scaling, rotation, homogeneous coordinates, composite matrices).
- **Conventions used here:** clipping window is an **axis-aligned rectangle** with corners (xwmin, ywmin) and (xwmax, ywmax). Region codes are written **T B R L**. Books differ in bit order, so follow your own textbook in the exam.
- These notes do not assume any particular university syllabus or textbook. Check your syllabus for exact depth.

Learning outcomes — after this topic you can…
1. Draw and explain the 2-D viewing pipeline and the role of every coordinate system.
2. Convert world coordinates to viewing coordinates and map a window to a viewport.
3. Explain clipping and perform point, line (Cohen–Sutherland) and polygon (Sutherland–Hodgman) clipping.
4. Solve numerical problems and avoid the usual mistakes.

1 Introduction: Seeing a Part of the World

A graphics scene can be much larger than the screen. We usually want to show **only one part** of it, in a chosen **area of the screen**. 2-D viewing is the set of steps that selects that part, maps it to the screen and discards everything outside.

DEFINITIONS

- **World coordinates (WC):** the coordinate system in which the whole scene is described.
- **Window:** the rectangular area of the **world** that we want to display.
- **Viewport:** the rectangular area of the **display device (screen)** where the window contents appear.
- **Clipping:** removing the parts of a picture that lie outside a given region (the clip window).

Real-world example	Window	Viewport
Google Maps on a phone	The map region you are looking at (changes as you pan/zoom)	The phone's map screen area
Mini-map in a video game	A larger region around the player	A small box in a corner of the screen
Split-screen multiplayer game	Each player's camera region	Left half and right half of the screen
CAD drawing zoom	The zoomed part of the drawing	The drawing area of the application
Overlapping GUI windows	—	Clipping stops a window from drawing over the one in front of it

IMPORTANT

Panning = moving the window over the world. **Zooming** = making the window smaller (zoom in) or larger (zoom out) while the viewport stays the same size.

2 The 2-D Viewing Pipeline

From the object's own coordinates to screen pixels

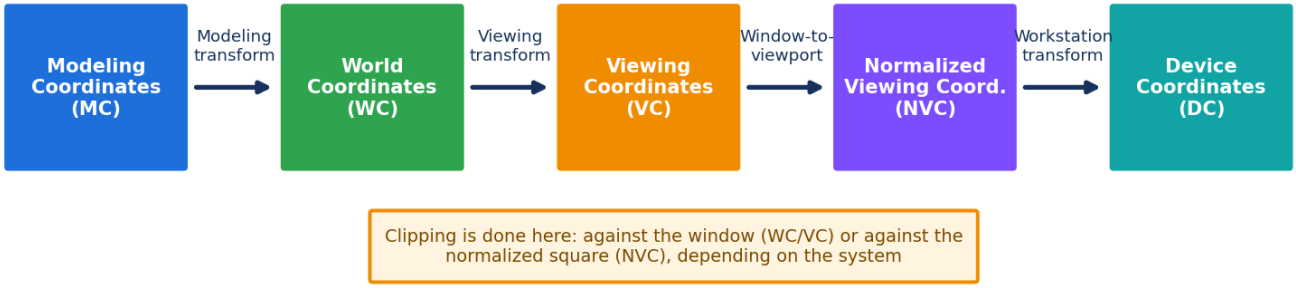


Figure 1: The 2-D viewing pipeline (as commonly presented in textbooks). Where clipping is applied can differ between systems.

Stage	Meaning	Typical question it answers
Modeling coordinates (MC)	Each object is described in its own local system	Where is a vertex relative to the object's centre?
World coordinates (WC)	All objects are placed in one common scene	Where is the object in the scene?
Viewing coordinates (VC)	A system aligned with the window (origin + orientation of the "camera")	Where is the point relative to the window?
Normalized viewing coordinates (NVC)	Device-independent system, often 0 to 1 (some systems use −1 to 1)	What fraction of the way across the window is the point?
Device coordinates (DC)	Actual pixel positions of one device	Which pixel is it?

WHY NORMALIZE?

Normalized coordinates make the picture **device independent**. The same normalized picture can be sent to different screens by a simple final scaling, so the program does not need to know the screen size.

WHERE TO CLIP?

Clipping early (against the window) means fewer points go through the later transformations. Clipping against the normalized square (a fixed, simple boundary) is also common. Both ideas appear in textbooks; the result is the same picture.

3 Viewing Coordinate Reference Frame

Sometimes the window is not parallel to the world axes (for example, a rotated map or a camera that is tilted). We then define a **viewing coordinate system** so that the window is aligned with its axes.

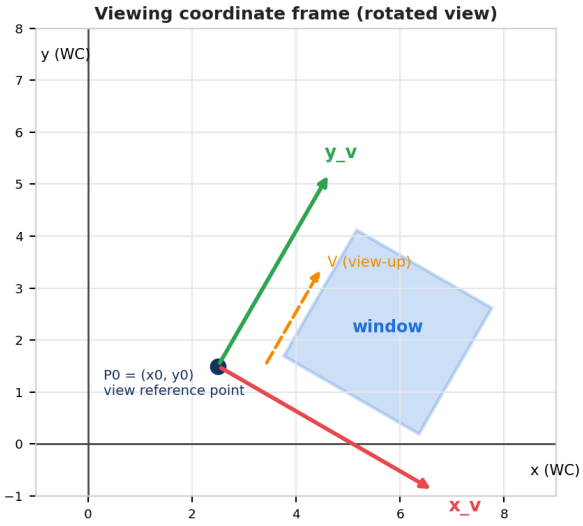


Figure 2: A viewing frame with origin P0 and view-up vector V. The window is a rectangle aligned with x_v and y_v.

DEFINITIONS

- **View reference point P0 = (x0, y0):** the origin of the viewing coordinate system, chosen in world coordinates.
- **View-up vector V:** a vector that gives the direction of the positive y_v axis.
- **Unit vectors:** v = V/|V| (the y_v direction) and u = (Vy, −Vx)/|V| (the x_v direction, perpendicular to v).

World → viewing transformation (same idea as changing the coordinate system in the previous notes): move P0 to the origin, then rotate so that u and v become the x and y axes.

```
M(WC -> VC) = R · T(-x0, -y0)

        | ux   uy   0 |
    R = | vx   vy   0 |        (rows are the unit vectors u and v)
        | 0    0    1 |
```

Example 1: World to viewing coordinates

P0 = (2, 1), V = (−1, 0), point P = (5, 3).
v = (−1, 0), u = (Vy, −Vx) = (0, 1). Translate: (5−2, 3−1) = (3, 2).
x_v = u · (3,2) = 2 and y_v = v · (3,2) = −3. **P in VC = (2, −3).**
Check: the axes here are the world axes rotated 90° CCW, which matches the answer from the transformations notes.

IMPORTANT

If V is parallel to the world y-axis (V = (0, 1)), then R = I and the viewing frame is just a translated copy of the world frame.

4 Window-to-Viewport Coordinate Transformation

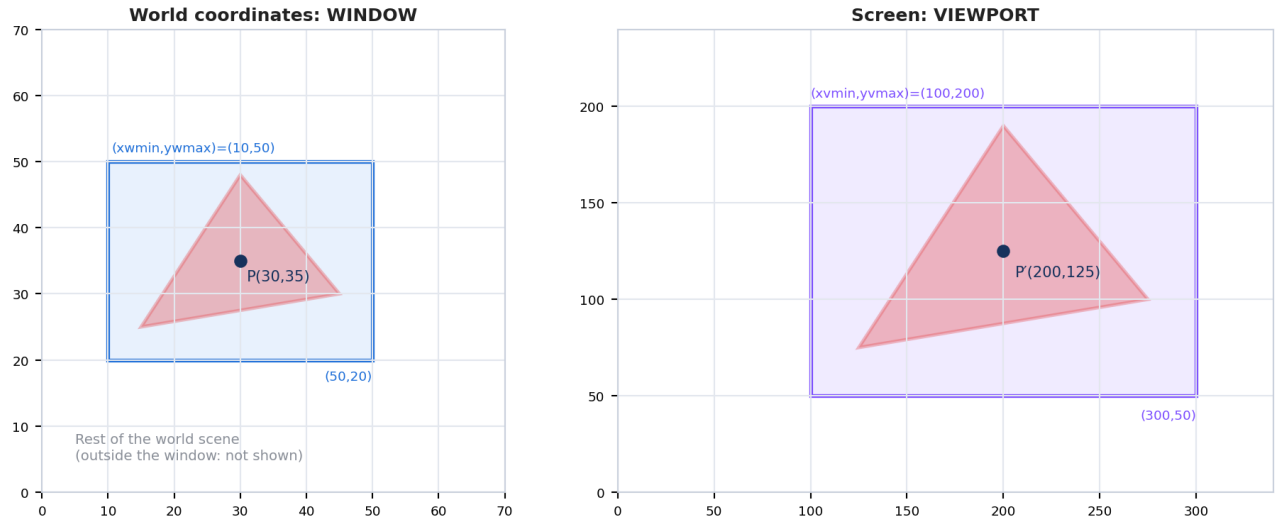


Figure 3: The window (left) is mapped to the viewport (right). The point and triangle keep the same *relative* position.

IDEA

A point must keep the same **relative position** inside the viewport as it had inside the window. This gives the equation:

```
(xv - xvmin) / (xvmax - xvmin)  =  (xw - xwmin) / (xwmax - xwmin)
(yv - yvmin) / (yvmax - yvmin)  =  (yw - ywmin) / (ywmax - ywmin)
```

Solving gives:

```
xv = xvmin + (xw - xwmin) · sx          sx = (xvmax - xvmin) / (xwmax - xwmin)
yv = yvmin + (yw - ywmin) · sy          sy = (yvmax - yvmin) / (ywmax - ywmin)
```

As a composite matrix (three steps, using the transformations you already know):

1. Translate the window corner to the origin: **T(−xwmin, −ywmin)**
2. Scale the window size to the viewport size: **S(sx, sy)**
3. Translate to the viewport corner: **T(xvmin, yvmin)**

```
M(window -> viewport) = T(xvmin, yvmin) · S(sx, sy) · T(-xwmin, -ywmin)
```

M as one 3×3 matrix

$$\begin{bmatrix} sx & 0 & xvmin - xwmin \cdot sx \\ 0 & sy & yvmin - ywmin \cdot sy \\ 0 & 0 & 1 \end{bmatrix}$$

Example 2: Mapping a point

Window: (xwmin, ywmin) = (10, 20), (xwmax, ywmax) = (50, 50). Viewport: (100, 50) to (300, 200). Map P(30, 35).
sx = (300−100)/(50−10) = 200/40 = 5, sy = (200−50)/(50−20) = 150/30 = 5.
xv = 100 + (30−10)·5 = 200, yv = 50 + (35−20)·5 = 125. **P′ = (200, 125)** (see Figure 3).

Example 3: Distortion

Window (0,0)–(10,10), viewport (0,0)–(200,100). sx = 20, sy = 10, so sx ≠ sy. The point (5,5) maps to **(100, 50)**, and a square in the window becomes a **rectangle** (stretched in x).

Case	Result
sx = sy	Shape is only scaled; no distortion
sx ≠ sy	Picture is stretched or squashed (distortion)
sx, sy > 1 or < 1	Window region is magnified or shrunk on screen (zoom effect)
Viewport bigger than window	Objects look larger

SCREEN Y-AXIS

Many screens have the origin at the **top-left** with y increasing **downwards**. Then a final flip (y_screen = H − y) is needed, as discussed in the coordinate-system part of the previous notes. Check the convention of your device or library.

	Window	Viewport
Where it lives	World coordinate space	Display device (or normalized) space
What it decides	*What* part of the scene is shown	*Where* and how big it appears
Changing it causes	Pan / zoom	Move / resize the picture on screen
Units	World units (metres, inches, ...)	Pixels (or 0 to 1 if normalized)

5 Viewing Functions

DEFINITION

Viewing functions are the routines of a graphics package that let a programmer **specify the window, the viewport and the view orientation** without writing the matrices by hand. The package builds and applies the transformation matrices internally.

Purpose	Generic idea	Examples (names vary by package)
Set the window	Give xwmin, xwmax, ywmin, ywmax	OpenGL 2-D: `gluOrtho2D(left, right, bottom, top)`
Set the viewport	Give position and size on the screen	OpenGL: `glViewport(x, y, width, height)`
Set the view orientation	Give P0 and the view-up vector V	Some textbooks use PHIGS-style routines, e.g. `evaluateViewOrientationMatrix`
Combine window and viewport	Build the window-to-viewport matrix	Textbook routines such as `evaluateViewMappingMatrix`; packages do it internally
Choose / switch the view	Select one of several stored views	e.g. `setViewRepresentation`, `setViewIndex` in PHIGS-style packages

NOTE

Function names are not universal. Use the names given in **your prescribed textbook or lab manual**. The **concepts** (window, viewport, orientation) are what matter in theory answers.

ASPECT RATIO

To avoid distortion, keep the viewport aspect ratio (width/height) equal to the window aspect ratio. This is why resizing a program window can stretch a drawing if the code does not update the viewport or window.

6 Clipping Operations

DEFINITION

Clipping is the process of identifying which parts of a picture lie **inside** a specified region (the **clip window**) and which lie **outside**, and removing the outside parts. The clip window is most often a rectangle.

Clipping type	Object clipped	Typical algorithm in this unit
Point clipping	Single points (pixels, particles)	Four inequality tests
Line clipping	Line segments	Cohen–Sutherland
Polygon (area) clipping	Filled areas, polygons	Sutherland–Hodgman
Curve / text clipping	Curves, characters	Other methods (not covered here)

Why clip? (1) Avoid drawing outside the display area. (2) Save time by not processing hidden parts. (3) Keep drawing inside windows (GUI). (4) Avoid errors from coordinates outside the device range.

7 Point Clipping

DEFINITION

A point P(x, y) is **inside** the clip window if and only if all four conditions hold:

```
xwmin  <=  x  <=  xwmax
ywmin  <=  y  <=  ywmax
```

Example 4

Window (10,10)–(50,40). P1(30,20): inside. P2(5,20): x < 10, outside. P3(50,40): on the boundary, treated as inside. P4(30,45): y > 40, outside.

REAL-WORLD

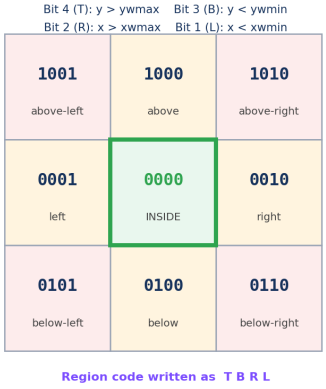
A particle system (sparks, rain) draws only particles whose position is inside the screen.

IMPORTANT

Boundary points (equality) are normally treated as **inside** (≤ and ≥). State this assumption in exams.

8 Line Clipping: Cohen–Sutherland Algorithm

Idea: quickly accept or reject most lines using **region codes** (outcodes), and compute intersections only for the lines that need them.



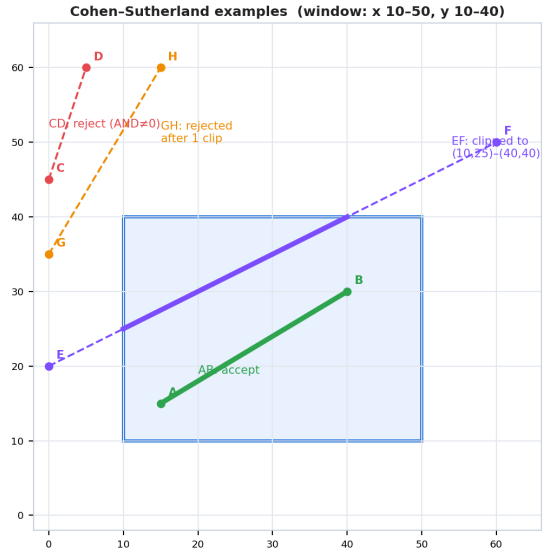


Figure 4 and 5: Nine regions with 4-bit codes (left). Four example lines (right): accepted, rejected, clipped and rejected after one clip.

REGION CODE

Each endpoint gets a 4-bit code **T B R L**. A bit is set to 1 when:
L (bit 1): x < xwmin • **R** (bit 2): x > xwmax • **B** (bit 3): y < ywmin • **T** (bit 4): y > ywmax.
A point inside the window has code **0000**.

Algorithm steps

1. Find the region codes **code1** and **code2** of the two endpoints.
2. **Trivial accept:** if code1 = 0000 **and** code2 = 0000 (equivalently code1 OR code2 = 0000), the whole line is inside. Draw it.
3. **Trivial reject:** if code1 AND code2 ≠ 0000, both endpoints are on the same outer side. Discard the line.
4. Otherwise, take an endpoint that is outside (non-zero code). Check its bits in a fixed order (e.g. L, R, B, T) and find the intersection with the boundary for the **first set bit**.
5. Replace that endpoint by the intersection point, recompute its code, and go back to step 2.

```
m = (y2 - y1) / (x2 - x1)                    <- slope
Vertical boundary   (x = X):  y = y1 + m · (X - x1)
Horizontal boundary (y = Y): x = x1 + (Y - y1) / m
```

KEY FACTS

- A line needs **at most four** boundary clips (one for each window edge).
- A **vertical** line (x1 = x2) never needs the slope for a vertical boundary: if it is outside left/right, both endpoints share that bit and it is trivially rejected. A **horizontal** line is handled the same way for top/bottom.
- AND ≠ 0 means reject. **AND = 0 does not mean accept**: the line may be partly inside, or fully outside around a corner.

Worked examples (window: xwmin = 10, ywmin = 10, xwmax = 50, ywmax = 40)

Line	Codes (T B R L)	Decision / steps	Result
A(15,15)–B(40,30)	0000 and 0000	Both inside: trivial accept	Draw A–B
C(0,45)–D(5,60)	1001 and 1001	AND = 1001 ≠ 0: trivial reject	Discard
E(0,20)–F(60,50)	0001 and 1010	AND = 0000, needs clipping. m = 30/60 = 0.5 (details below)	(10,25)–(40,40)
G(0,35)–H(15,60)	0001 and 1000	AND = 0000. Clip G at x = 10: y = 35 + (5/3)(10) = 51.67, point (10, 51.67) has code 1000. Now both codes 1000: reject	Discard

Example 5: Full steps for E(0,20)-F(60,50)

m = (50−20)/(60−0) = 0.5.
Step 1: E has code 0001 (left). Clip with x = 10: y = 20 + 0.5(10−0) = 25. New E = (10, 25), code 0000.
Step 2: F has code 1010. First set bit in the order L, R, B, T is R. Clip with x = 50: y = 25 + 0.5(50−10) = 45. New F = (50, 45), code 1000 (above).
Step 3: F has code 1000. Clip with y = 40: x = 10 + (40−25)/0.5 = 40. New F = (40, 40), code 0000.
Both codes 0000: accept. Clipped line = **(10,25) to (40,40)**. Check: y = 20 + 0.5x gives y(40) = 40. ✓

Advantages	Limitations
Simple to understand and implement	Clip window must be an **axis-aligned rectangle**
Very fast when most lines are trivially accepted or rejected	Can compute several intersections (and recompute codes) for one line
Only integer-like comparisons for the trivial tests	Not the most efficient for lines that need clipping

Algorithm	Main idea	Window type	Remark
Cohen–Sutherland	Region codes, repeated edge clipping	Rectangle	Simple; good if most lines are trivial
Liang–Barsky	Parametric line equation, inequalities on parameter t	Rectangle	Fewer intersection calculations
Cyrus–Beck	Parametric clipping using edge normals	Any convex polygon	General convex window
Nicholl–Lee–Nicholl	Region analysis around the window to avoid repeated clips	Rectangle (2-D)	Fewer comparisons and divisions

Only Cohen–Sutherland is in this unit's topic list; the other algorithms are shown for comparison. Check whether your syllabus includes them.

9 Polygon Clipping: Sutherland–Hodgman Algorithm

WHY NOT CLIP EDGE BY EDGE WITH LINE CLIPPING?

If each polygon edge is clipped separately, the result is a set of separate lines, not a **closed** polygon. We need extra edges along the window boundary so that the clipped polygon can still be **filled**.

SUTHERLAND–HODGMAN IDEA

Clip the polygon against **one window edge at a time** (infinite boundary line). The output vertex list of one edge becomes the **input list** of the next edge. After all four edges, the list is the clipped polygon.

Processing one boundary

Go around the polygon edge by edge. For each edge from vertex **S** (previous) to vertex **P** (current), compare them with the boundary. **I** is the intersection of edge S–P with the boundary. Include the closing edge (last vertex → first vertex).

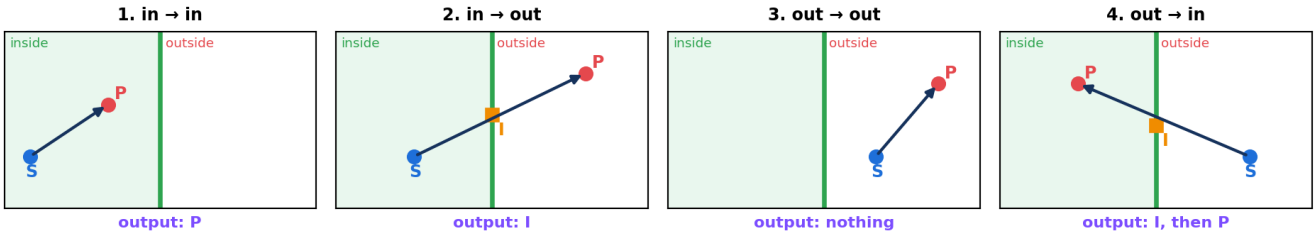


Figure 6: The four cases for edge S → P against one clip boundary (green = inside side).

Case	S	P	Output to the new list
1	inside	inside	P
2	inside	outside	I (intersection)
3	outside	outside	nothing
4	outside	inside	I, then P

```
for each window edge (left, right, bottom, top):
    new_list = empty
    S = last vertex of current list
    for each vertex P of current list:
        apply the four cases for (S, P) and add output to new_list
        S = P
    current list = new_list
```

Example 6: Full worked example

Window: 0 ≤ x ≤ 10, 0 ≤ y ≤ 10. Polygon: V1(−2,5), V2(6,−3), V3(13,4), V4(5,12). Clip in the order left, right, bottom, top.
Left (x = 0): V4→V1 crosses at (0,7); V1→V2 crosses at (0,3). List: (0,7), (0,3), (6,−3), (13,4), (5,12).
Right (x = 10): V2→V3 crosses at (10,1); V3→V4 crosses at (10,7). List: (0,7), (0,3), (6,−3), (10,1), (10,7), (5,12).
Bottom (y = 0): (0,3)→(6,−3) crosses at (3,0); (6,−3)→(10,1) crosses at (9,0). List: (0,7), (0,3), (3,0), (9,0), (10,1), (10,7), (5,12).
Top (y = 10): (5,12)→(0,7) crosses at (3,10); (10,7)→(5,12) crosses at (7,10). **Final: (3,10), (0,7), (0,3), (3,0), (9,0), (10,1), (10,7), (7,10)**, an 8-sided polygon.

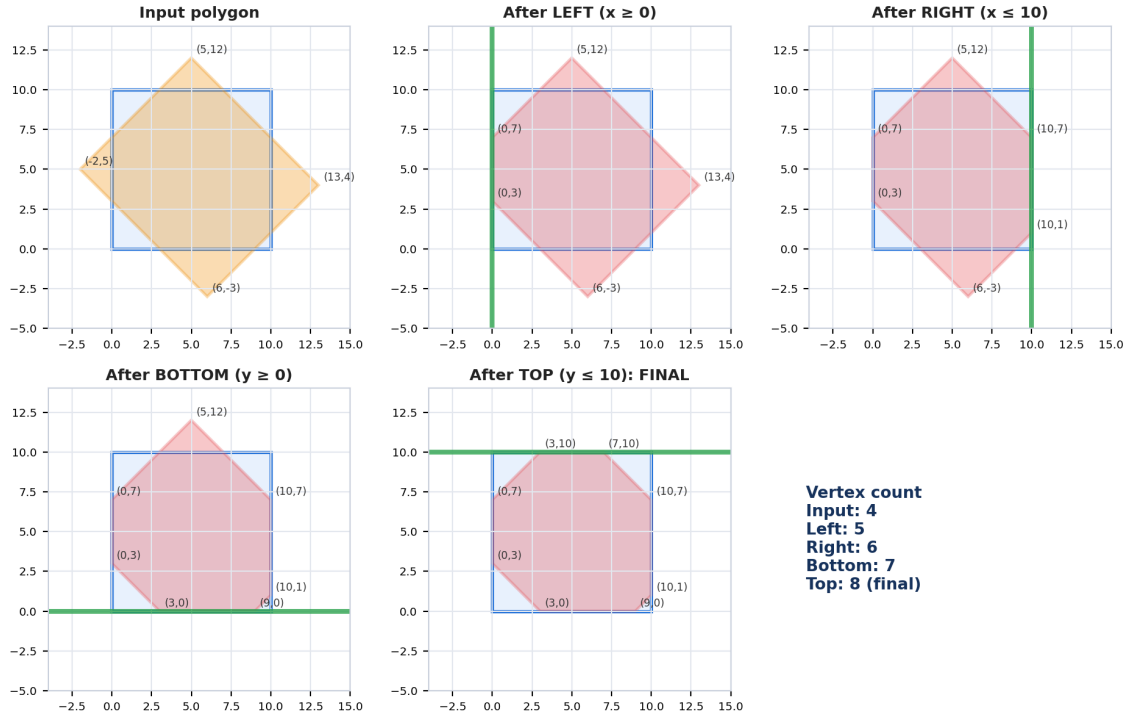


Figure 7: The polygon after each clipping stage (green line = boundary being used). The vertex count grows from 4 to 8.

Aspect	Cohen–Sutherland (line)	Sutherland–Hodgman (polygon)
Object clipped	Line segments	Polygons (closed areas)
Core idea	4-bit region codes; trivial accept/reject	Clip against one boundary at a time; four cases for each edge
Output	A shorter line segment or nothing	A new vertex list (can have more vertices)
Window	Rectangle	Convex clip region (rectangle in this unit)
Order of window edges	Fixed order of bits, recomputed codes	Any fixed order; output of one is input of next

LIMITATIONS

- Works correctly for a **convex** polygon against a convex window.
- For a **concave** polygon, the result may contain extra overlapping edges along the window boundary (a degenerate polygon) when the clipped parts are separate. Such cases need extra handling or a different algorithm.
- The number of output vertices can be larger than the input.

10 Common Student Mistakes

Avoid these

1. Mixing up **window** (in the world) and **viewport** (on the screen).
2. Writing sx = window size / viewport size. It is **viewport size / window size**.
3. Forgetting to subtract xwmin or to add xvmin when mapping a point.
4. Ignoring distortion when sx ≠ sy, or the screen y-axis direction.
5. Using different region-code bit orders in the same answer.
6. Accepting a line when **only one** endpoint has code 0000.
7. Rejecting when AND = 0000, or accepting when AND ≠ 0000 (the two rules are not the same).
8. Not **recomputing the code** after replacing an endpoint with an intersection.
9. Using the wrong intersection formula: vertical boundary uses y = y1 + m(X − x1), horizontal uses x = x1 + (Y − y1)/m.
10. Clipping polygon edges with a line-clipping method and leaving an **open** polygon.
11. In Sutherland–Hodgman: forgetting the closing edge (last → first vertex), or not feeding the output of one boundary into the next.
12. Using the wrong inside/outside side for a boundary (e.g. “inside” for the right edge is x ≤ xwmax).
13. Expecting perfect output for a **concave** polygon.

11 5-Minute Classroom Activity: “Clip It!”

Task (work in pairs)

1. Window: xwmin = 0, ywmin = 0, xwmax = 10, ywmax = 10. Draw it on graph paper. (30 s)
2. For each line, write the **region codes (T B R L)** and decide: accept, reject or clip. (3 min)

L1: (2,3)–(8,7) L2: (−3,12)–(−1,15) L3: (−4,2)–(6,12)

3. For the line that needs clipping, find the new endpoints. (1 min)
4. Swap papers and check each other. (30 s)

Line	Codes	Decision	Result
L1 (2,3)–(8,7)	0000, 0000	Trivial accept	(2,3)–(8,7)
L2 (−3,12)–(−1,15)	1001, 1001	AND ≠ 0: trivial reject	Nothing drawn
L3 (−4,2)–(6,12)	0001, 1000	AND = 0: clip. m = 1. Left: x = 0 gives y = 6. Top: y = 10 gives x = 4	**(0,6)–(4,10)**

Learning outcome

Most lines are decided by the codes alone; only some need intersection calculations.

12 Interview Questions

#	Question	Key answer
1	Difference between window and viewport?	Window = region of the world to display; viewport = region of the screen where it is shown.
2	What is the viewing pipeline?	MC → WC → VC → NVC → DC through successive transformations, with clipping.
3	Why use normalized coordinates?	Device independence: one normalized picture can be mapped to any screen.
4	How is the window-to-viewport matrix built?	$T(xvmin,yvmin)\cdot S(sx,sy)\cdot T(-xwmin,-ywmin)$.
5	When does distortion occur in the mapping?	When $sx \neq sy$ (aspect ratios of window and viewport differ).
6	What is the view-up vector?	Vector giving the direction of the positive y-axis of the viewing frame.
7	What is clipping and why is it needed?	Removing parts outside a region to save work and avoid drawing outside the display.
8	Explain Cohen–Sutherland in 30 seconds.	4-bit codes; accept if both 0; reject if AND ≠ 0; otherwise clip against an edge and repeat.
9	Does AND = 0000 mean the line is visible?	No. It means the line cannot be trivially rejected; it may still be fully outside.
10	Maximum number of clips per line in Cohen–Sutherland?	Four, one per window edge.
11	Why can't we clip polygon edges one by one?	The result would not be a closed polygon; boundary edges must be added.
12	Which polygons does Sutherland–Hodgman handle correctly?	Convex polygons against a convex window; concave ones can give extra edges.
13	How does a game implement a mini-map?	A second window/viewport pair: a large world region drawn in a small screen area.

13 University Examination Questions

Short answer (2–3 marks)

1. Define window and viewport.
2. Draw the 2-D viewing pipeline.
3. What is the view-up vector?
4. Write the window-to-viewport equations.
5. What is clipping? Name its types.
6. Write the point-clipping conditions.
7. What is a region code in Cohen–Sutherland? Give the code for the inside region.
8. State the trivial accept and trivial reject conditions.

Medium answer (5–6 marks)

1. Explain the viewing coordinate reference frame and derive the world-to-viewing matrix.
2. Derive the window-to-viewport transformation and express it as a composite matrix.
3. Window: (20,30)–(60,70). Viewport: (0,0)–(1,1). Map P(40,50). *[Answer: (0.5, 0.5)]*

4. Window: (0,0)–(100,50). Viewport: (0,0)–(800,600). Find sx, sy and map (25,10). What happens to the shape? *[Answer: sx = 8, sy = 12, point (200,120); picture is stretched more in y]*
5. Explain the Cohen–Sutherland algorithm with a diagram of the region codes.
6. Clip line (0,20)–(60,50) against the window (10,10)–(50,40) using Cohen–Sutherland. *[Answer: (10,25)–(40,40)]*
7. Explain the four cases of Sutherland–Hodgman with a diagram.
8. List the limitations of Sutherland–Hodgman polygon clipping.

14 Five Long-Answer Questions

DEFINITION

1. Explain the 2-D viewing pipeline in detail. Describe the role of modeling, world, viewing, normalized and device coordinates, and where clipping fits.
2. Define the viewing coordinate reference frame. Derive the transformation from world coordinates to viewing coordinates and solve: P0 = (2,1), V = (−1,0), P = (5,3).
3. Derive the window-to-viewport transformation. Express it as a product of basic transformations and discuss distortion. Map P(30,35) for the window (10,20)–(50,50) and the viewport (100,50)–(300,200).
4. Explain the Cohen–Sutherland line-clipping algorithm with region codes, steps and a flowchart or pseudocode. Clip the line (0,20)–(60,50) against the window (10,10)–(50,40) and show every step.
5. Explain the Sutherland–Hodgman polygon-clipping algorithm. Clip the polygon (−2,5), (6,−3), (13,4), (5,12) against the window (0,0)–(10,10) and list the vertices after each boundary. Mention its limitations.

Answer key (numerical parts)

- **Q2:** P in VC = (2, −3).
- **Q3:** sx = sy = 5, P′ = (200, 125).
- **Q4:** clipped line (10,25)–(40,40).
- **Q5:** final polygon (3,10), (0,7), (0,3), (3,0), (9,0), (10,1), (10,7), (7,10).

15 Short Recap

Remember

- **Window** = region of the world. **Viewport** = region of the screen.
- Pipeline: **MC → WC → VC → NVC → DC**, with clipping applied at a suitable stage.
- World → viewing: **R · T(−x0, −y0)** using view reference point P0 and view-up vector V.
- Window → viewport: **xv = xvmin + (xw − xwmin)·sx**, same for y. sx ≠ sy gives distortion.
- **Point clipping:** four inequality tests.
- **Cohen–Sutherland:** codes T B R L; accept if both 0000; reject if AND ≠ 0000; else clip and repeat (max 4 clips).
- **Sutherland–Hodgman:** clip against one edge at a time; four cases (in-in: P; in-out: I; out-out: none; out-in: I, P). Best for convex polygons.